

- **¿El resultado de `.query()` es idéntico al de su filtro\_avanzado? ¿Por qué?**  
**Respuesta:** Sí, el resultado es exactamente idéntico. Esto se debe a que `.query()` es simplemente un método alternativo (*wrapper* de evaluación expresiva) para realizar la misma selección booleana sobre las filas del DataFrame. Evalúa exactamente las mismas condiciones lógicas subyacentes, devolviendo un conjunto idéntico de índices y datos.
- **¿Cuál de las dos formas les parece más clara para leer?**  
**Respuesta:** `.query()` suele ser más clara para leer, especialmente cuando la consulta involucra múltiples condiciones. En lugar de encadenar corchetes `[]`, paréntesis `()` y operadores bit a bit (`&`, `|`), `.query()` permite escribir la condición en un formato similar a un lenguaje natural o lenguaje SQL, lo que facilita la comprensión rápida del filtro.
- **¿Qué ventaja tiene usar `@` en lugar de escribir el valor directamente en el texto?**  
**Respuesta:** La principal ventaja es el dinamismo y la flexibilidad del código. Usar `@` permite inyectar variables de Python directamente dentro de la expresión del filtro. Esto evita escribir valores fijos (*hardcoding*), permite reutilizar la consulta con distintos umbrales calculados dinámicamente en el programa y reduce el riesgo de errores al modificar manualmente las cadenas de texto.

### Ejercicio 13: `.isin()` y `~` — Incluir y excluir categorías

- **¿La suma de filas incluidas + excluidas da exactamente el total? ¿Por qué siempre debería ser así?**  
**Respuesta:** Sí, la suma da exactamente el total de filas del DataFrame original. Esto debe ser siempre así porque la operación `.isin()` y su negación lógica mediante `~` forman una partición exhaustiva y mutuamente excluyente del conjunto de datos: una fila está contenida en la lista elegida o no lo está. La suma de ambas partes reconstituye el 100% de los elementos originales.
- **¿Qué ventaja tiene `.isin(['A','B','C'])` frente a escribir `== 'A' | == 'B' | == 'C'`?**  
**Respuesta:** La ventaja radica en la legibilidad, mantenibilidad y concisión del código. Escribir múltiples condiciones con `==` y `|` vuelve el código largo, repetitivo y propenso a errores de paréntesis. `.isin()` permite pasar directamente una lista de valores (incluso una lista generada dinámicamente en Python), manteniendo la instrucción limpia en una sola línea.
- **¿Cuándo usarían la versión con `~` en un análisis real?**  
**Respuesta:** Se utiliza en la etapa de limpieza de datos (*Data Cleaning*) para filtrar o remover registros no deseados. Por ejemplo, para descartar transacciones canceladas, excluir países fuera del alcance del estudio, o

eliminar categorías residuales ("Otros", "Desconocido", etc.) sin tener que especificar manualmente todas las categorías válidas que queremos conservar.

#### Ejercicio 14: `.value_counts()`, `.unique()` y `.nunique()`

- **¿Cambia la distribución de categorías entre el DataFrame completo y el filtrado? ¿Qué dice eso?**

**Respuesta:** Sí, la distribución cambia significativamente. Al aplicar un filtro enfocado (por ejemplo, seleccionar películas que contengan "Drama" en su género), se restringe el universo de análisis a un subconjunto específico. Esto muestra cómo ciertos géneros secundarios coexisten con el género principal o cómo se sesgan las frecuencias cuando se aíslan segmentos de datos concretos.

- **¿Hay alguna categoría que desapareció completamente al aplicar el filtro?**

**Respuesta:** Sí, todas aquellas categorías o géneros principales que no cumplieron la condición del filtro parcial (por ejemplo, aquellos géneros que no contenían la palabra "Drama" o que no superaron el umbral de votos establecido) desaparecieron por completo en el DataFrame filtrado.

- **¿`value_counts()` y `groupby().count()` dan el mismo resultado? ¿Cuándo usarían cada uno?**

**Respuesta:**

- Dan la misma información en cuanto al conteo de elementos por categoría.
- `.value_counts()` es más directo y conveniente para la exploración rápida de una sola columna, ya que devuelve los conteos ordenados automáticamente de mayor a menor y permite calcular porcentajes directamente con `normalize=True`.
- `groupby().count()` se prefiere en análisis estructurados de múltiples columnas o cuando se necesitan realizar varias agregaciones avanzadas simultáneas (como contar, sumar y promediar sobre múltiples grupos a la vez).

#### Ejercicio 15: Exportar a CSV + Heatmap de correlación

- **¿Qué par de columnas tiene la correlación más alta? ¿Tiene sentido con sus datos?**

**Respuesta:** (Nota: Ajusta los nombres de variables según las columnas numéricas exactas de tu dataset). Por ejemplo, en datasets de IMDb, las variables numéricas como `num_votes` (número de votos) suelen correlacionarse de forma positiva moderada/alta con variables de popularidad o presupuesto. Tiene sentido conceptual ya que las películas

más vistas o populares acumulan un volumen sustancialmente mayor de votaciones por parte de los usuarios.

- **¿Qué significa una correlación cercana a 1? ¿Y cercana a 0? ¿Y negativa?**

**Respuesta:**

- **Cercana a 1:** Correlación positiva fuerte. Significa que cuando una variable aumenta, la otra tiende a aumentar en una proporción similar.
- **Cercana a 0:** Ausencia de relación lineal. Las variaciones en una variable no entregan información sobre los cambios en la otra.
- **Negativa (cercana a -1):** Correlación inversa fuerte. A medida que una variable aumenta, la otra disminuye.

- **¿Por qué usamos el DataFrame completo para calcular la correlación y no el filtrado?**

**Respuesta:** Para evitar el **sesgo de selección** (*selection bias*) o la restricción de rango (*range restriction*). Si calculamos la correlación en un subconjunto filtrado (por ejemplo, solo películas con un rango específico de votos o de un solo género), estamos recortando artificialmente la variabilidad de las variables. Calcular la matriz sobre el DataFrame completo refleja la estructura general de relaciones entre las variables numéricas de todo el universo de datos analizado.